

ITEC322 Information Technology Project Part B

Principles of Software Engineering

Assoc/Prof Walayat Hussain

Email: [walayat.hussain@acu.edu.au](mailto:walayathussain@acu.edu.au)

Office: 532.6.03

Consulting: Mondays 11-12

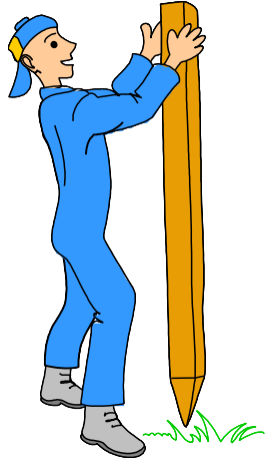
Software is Important

- ⚙ The economies of ALL developed nations are dependent on software
- ⚙ More and more systems are software controlled
- ⚙ Expenditure on software represents a significant fraction of GNP in all developed countries

Software is Complex

- ⚙️ Complex * complicated
- ⚙️ Complex = composed of many simple parts (related to one another)
- ⚙️ Complicated = not well understood, or explained

Complexity Example: Scheduling Fence Construction Tasks



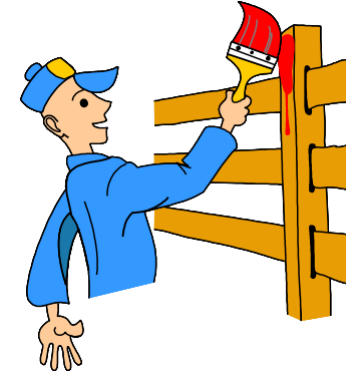
Setting posts
[3 time units]



Cutting wood
[2 time units]



Nailing
[2 time units for unpainted;
3 time units otherwise]



Painting
[5 time units for uncut wood;
4 time units otherwise]

Setting posts < Nailing, Painting

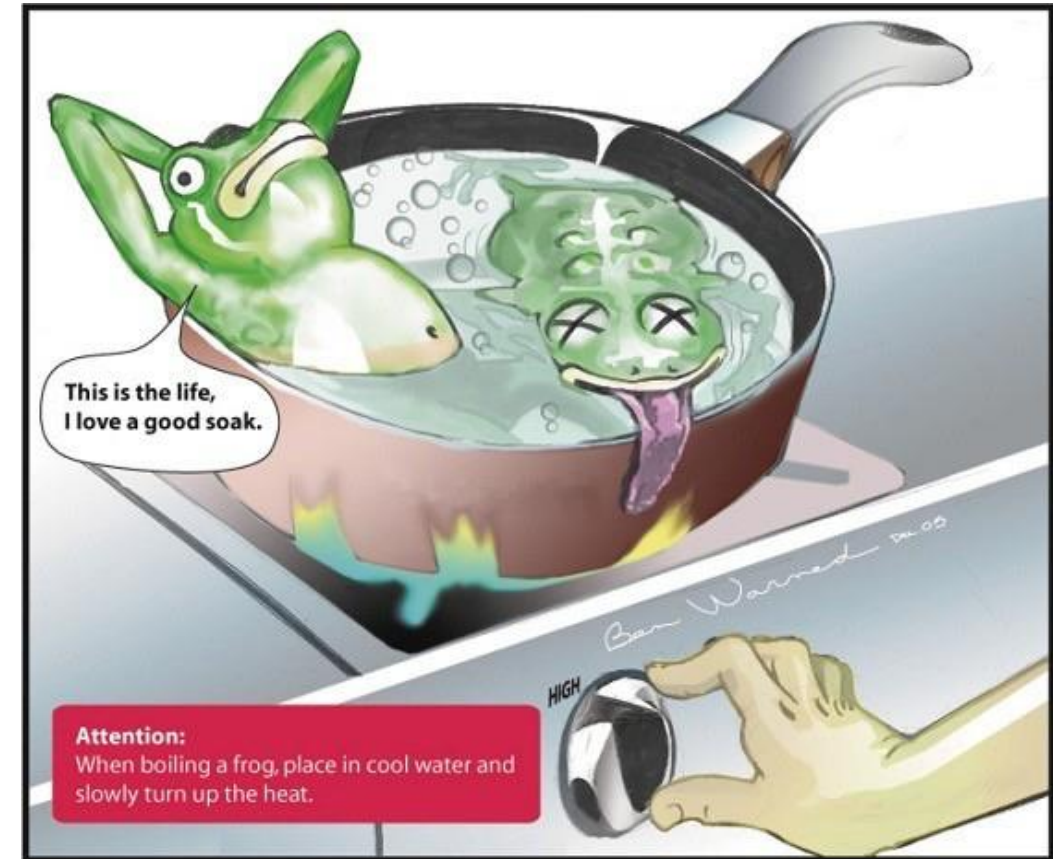
Cutting < Nailing

...shortest possible completion time = ?

[$\Rightarrow$ “simple” problem, but hard to solve without a pen and paper]

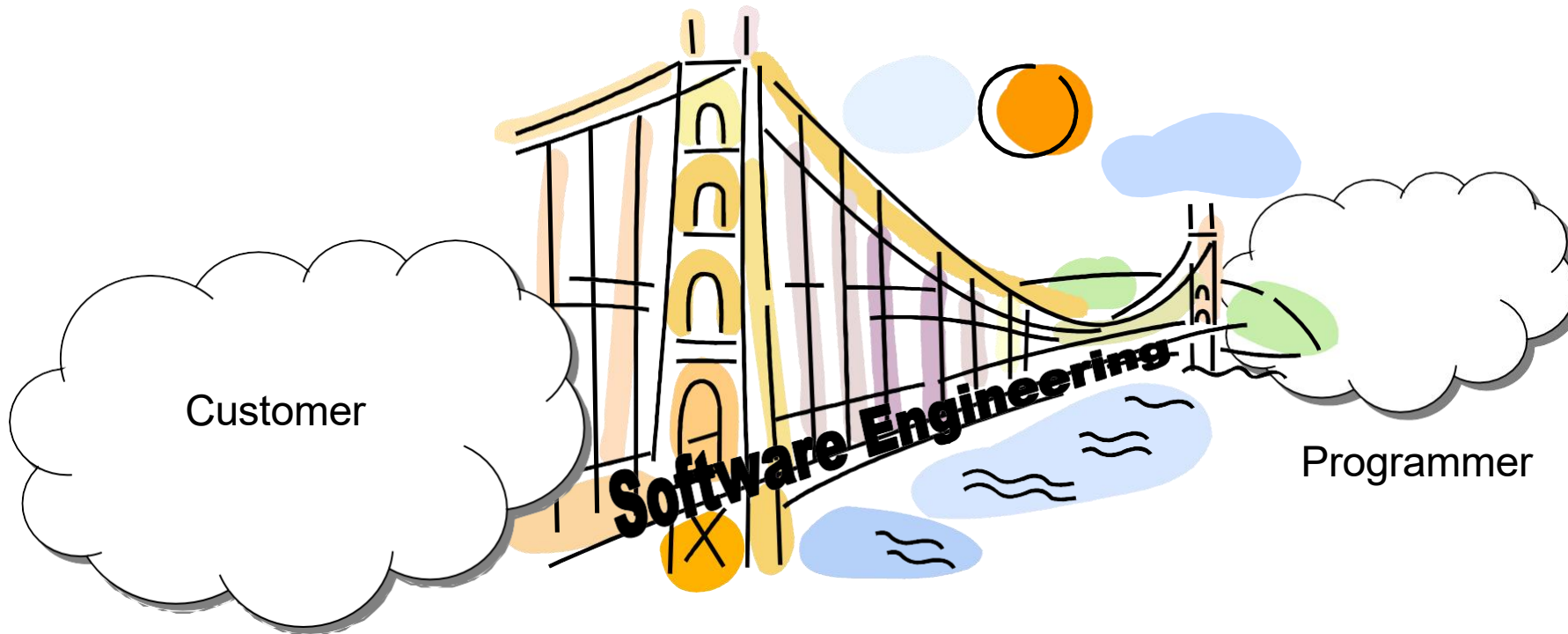
The Frog in Boiling Water

- ⚙ Small problems tolerate complacency—lack of immediate penalty leads to inaction
- ⚙ Negative feedback accumulates subtly and by the time it becomes painful, the problem is too big to address
- ⚙ Frog in gradually heated water analogy:
 - ☞ The problem with little things is that none of them is big enough to scare you into action, but they keep creeping up and by the time you get alarmed the problem is too difficult to handle
 - ☞ Consequently, “design smells” accumulate, “technical debt” grows, and the result is “software rot”

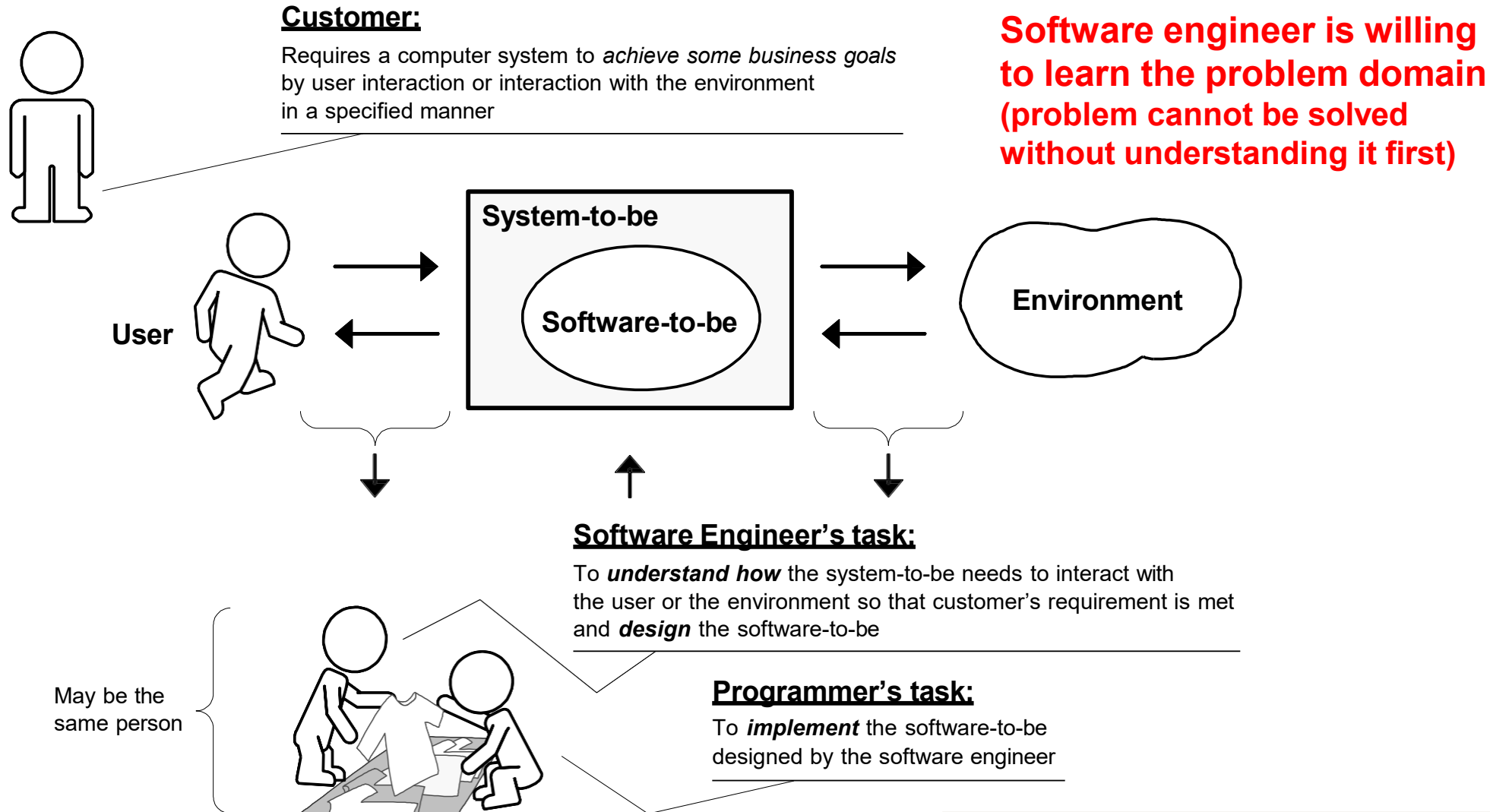


The Role of Software Engineering

A bridge from customer needs to programming implementation

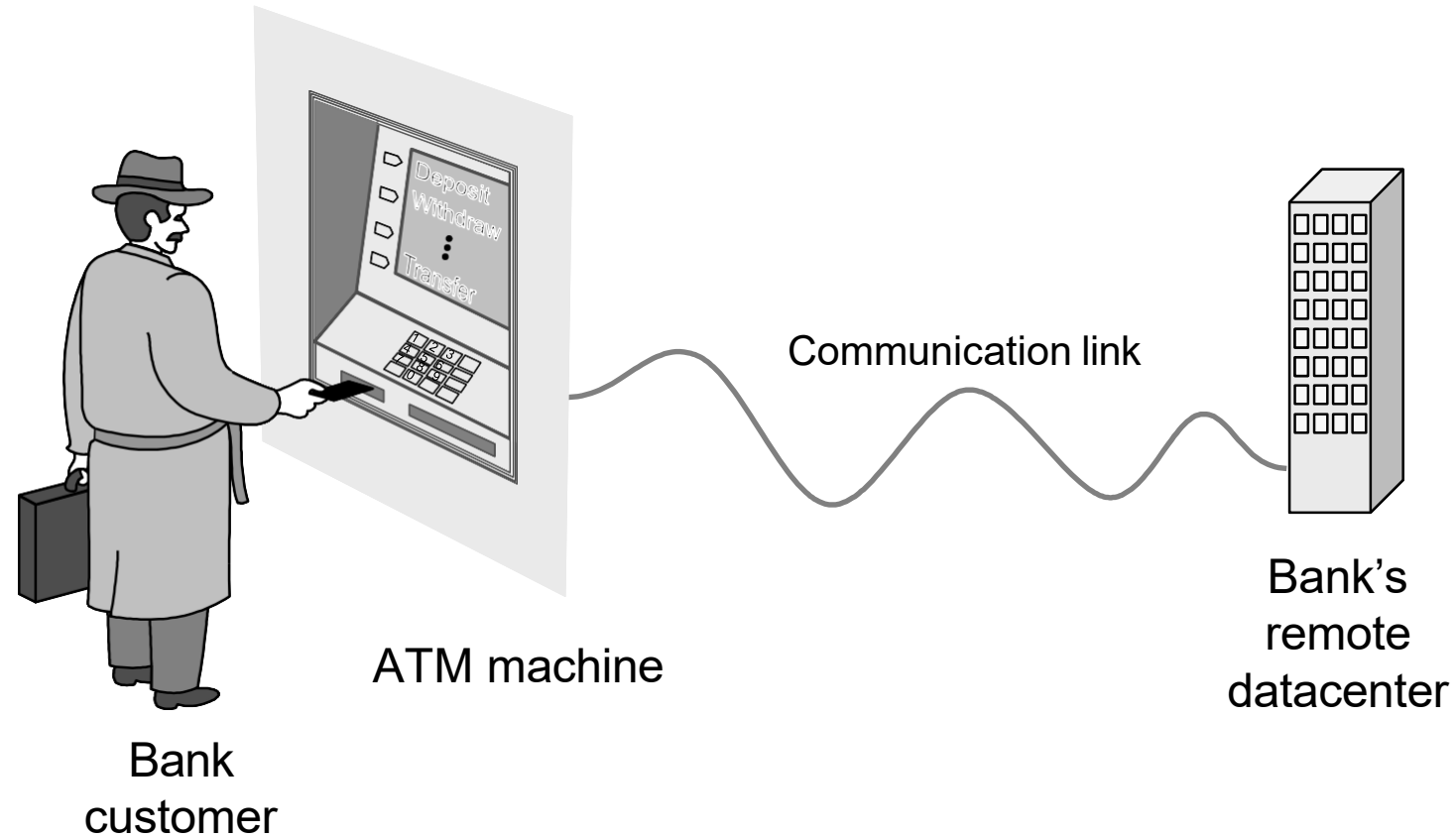


First law of software engineering



Example: ATM

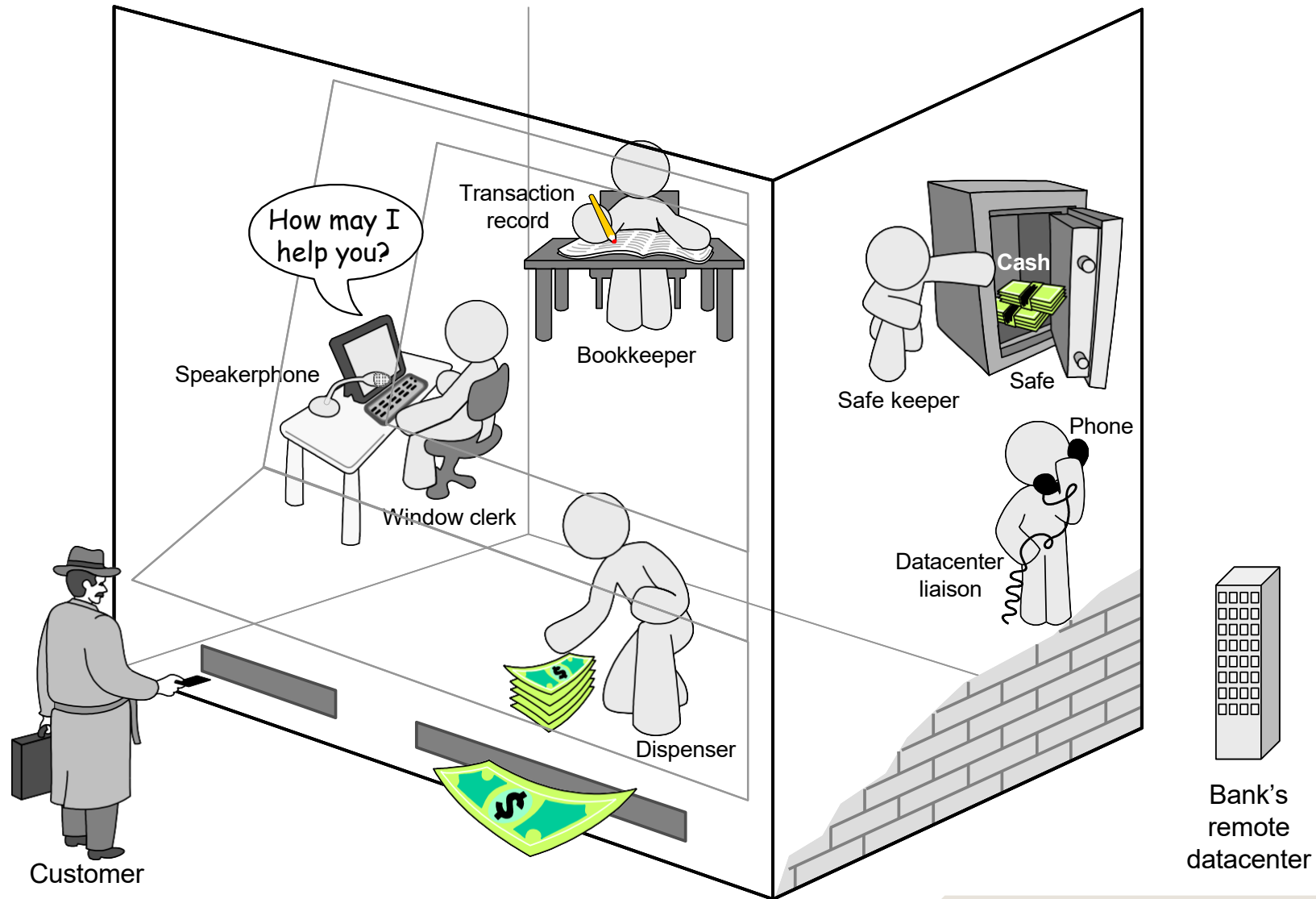
Understanding the money-machine problem:



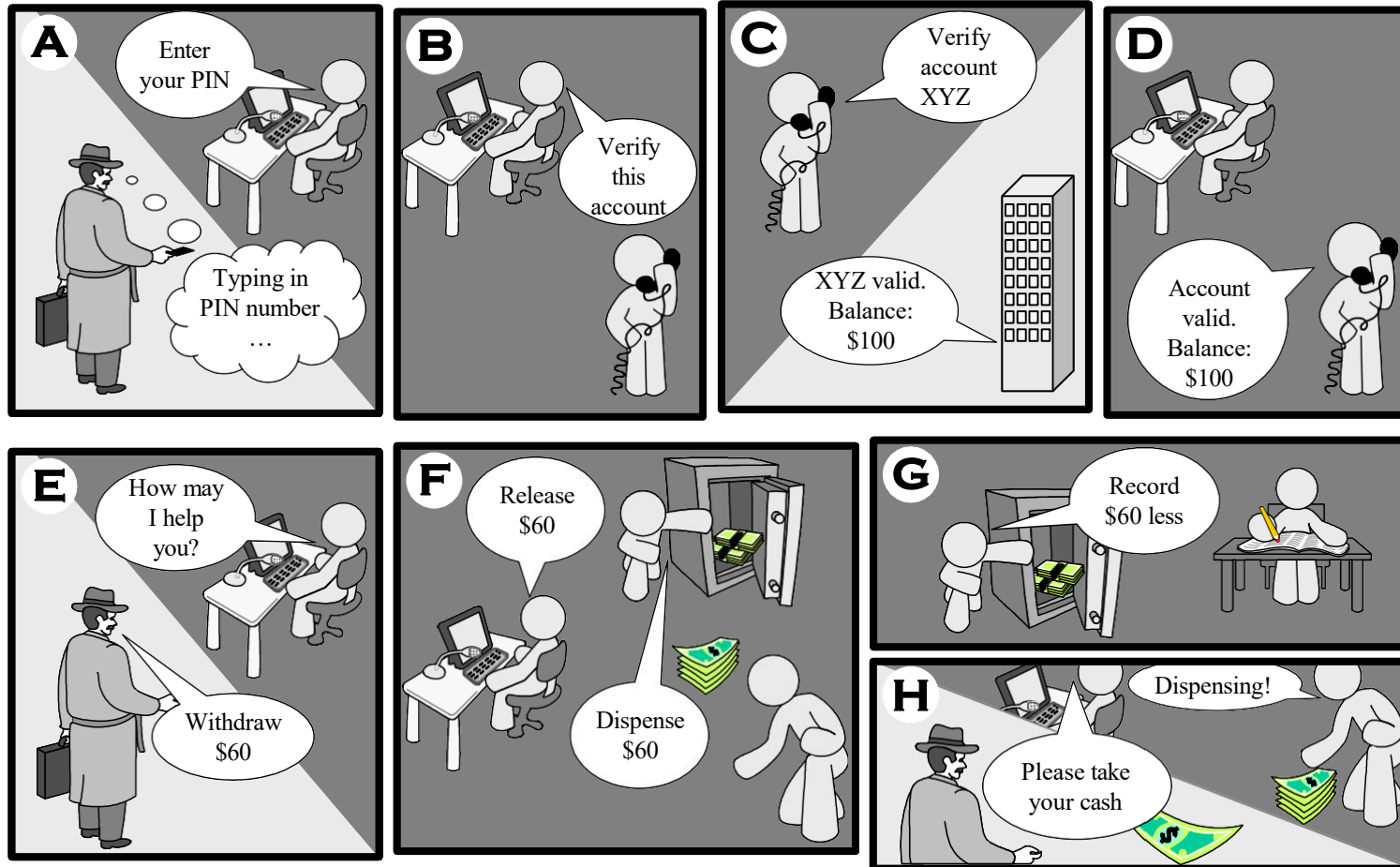
Understanding the Problem

- ⚙ Identify logical parts of the system that each solves a part of the problem
- ⚙ Easiest done with the help of a domain expert who already knows the steps in the process (“how it is currently done”)
- ⚙ Result: A Model of the Problem Domain (or “domain model”)

Domain Model



How ATM Works (Storyboarding)



Software Engineering Blueprints

- ⚙ Specifying software problems and solutions is like storyboarding (the ATM example)
- ⚙ Unfortunately, most of us are not artists, so we will use something less exciting:
UML symbols
- ⚙ However ...

Second Law of Software Engineering

⚙️ Software should be written for people first

- 👉 (Computers run software, but hardware quickly becomes outdated)
- 👉 Useful + good software lives long
- 👉 To nurture software, people must be able to understand it

What is Software Engineering?

- ⚙ Software engineering is an engineering discipline **that is concerned with all aspects of software production** from the early stages of system specification through to maintaining the system after it has gone into use
- ⚙ Engineering discipline
 - 👉 Using appropriate **theories and methods** to solve problems bearing in mind organizational and financial constraints
- ⚙ All aspects of software production
 - 👉 Not just technical process of development. Also project management and the development of tools, methods etc. to support software production

Essential attributes of good software

Product characteristic	Description
Maintainability	Software should be written in such a way so that it can evolve to meet the changing needs of customers . This is a critical attribute because software change is an inevitable requirement of a changing business environment.
Dependability and security	Software dependability includes a range of characteristics including reliability, security and safety . Dependable software should not cause physical or economic damage in the event of system failure. Malicious users should not be able to access or damage the system.
Efficiency	Software should not make wasteful use of system resources such as memory and processor cycles. Efficiency therefore includes responsiveness, processing time, memory utilisation, etc.
Acceptability	Software must be acceptable to the type of users for which it is designed. This means that it must be understandable, usable and compatible with other systems that they use.

Importance of software engineering

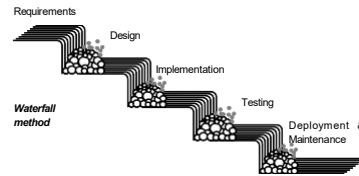
- ⚙ It is usually **cheaper, in the long run**, to use software engineering methods and techniques for software systems rather than just write the programs as if it was a personal programming project. For most types of system, **the majority of costs are the costs of changing the software after it has gone into use**
- ⚙ Software engineering is concerned with cost-effective software development

Software Development Methods

⚙️ Feynman Problem-Solving Algorithm

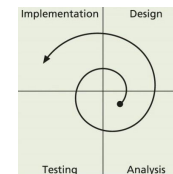
- 👉 Write down the problem
- 👉 think very hard
- 👉 write down the answer

⚙️ Waterfall



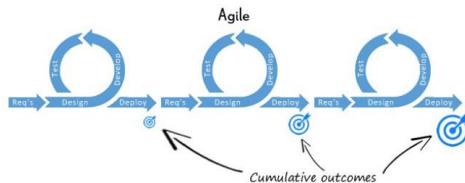
[What is the Waterfall Model and How Does it Work?](#)

⚙️ Spiral: Iterative + Incremental



[Spiral Process - Georgia Tech - Software Development Process](#)

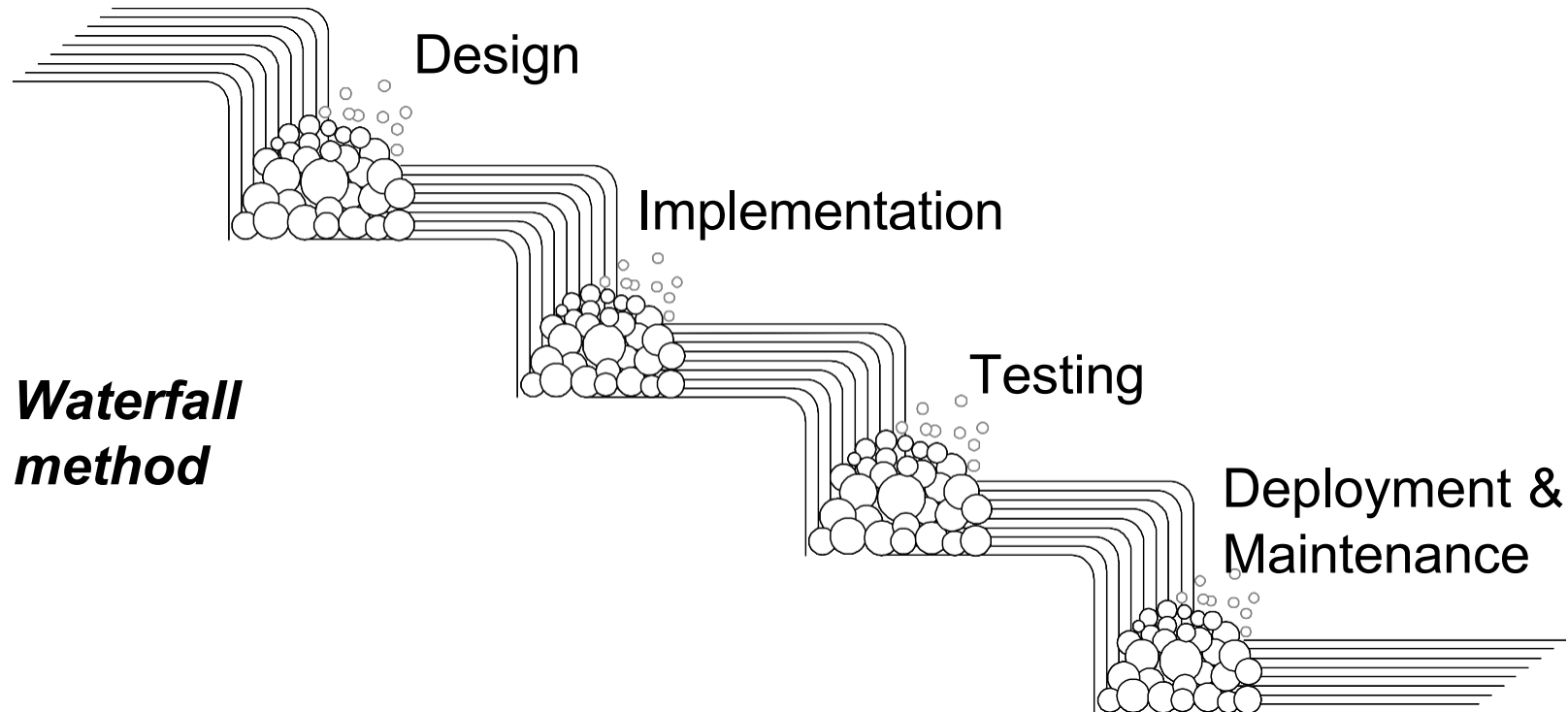
⚙️ Agile



[What Is Agile Methodology?](#)

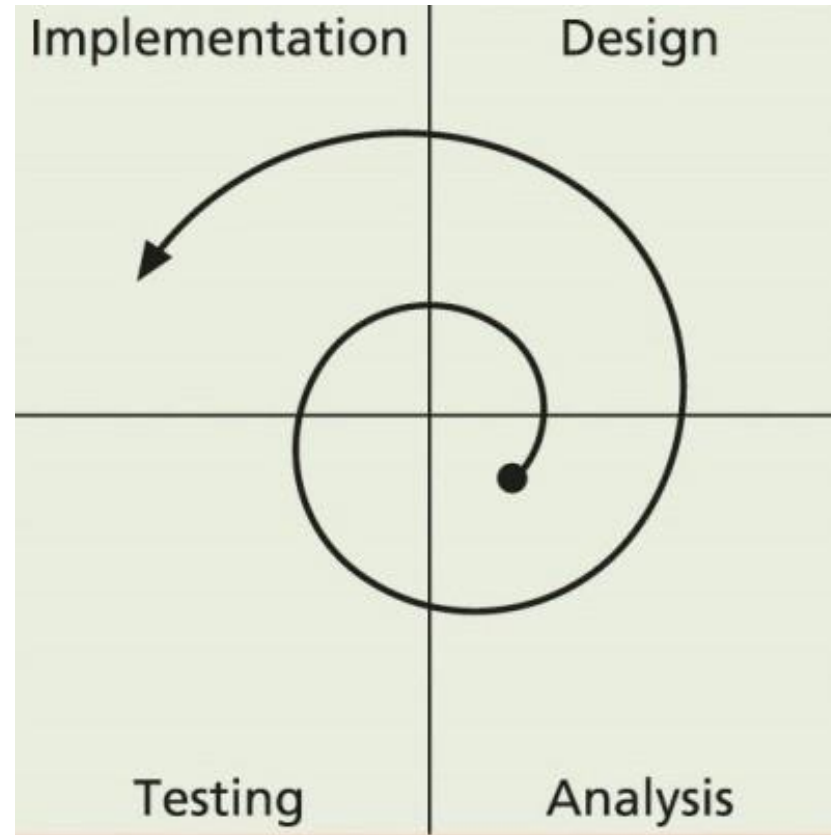
Waterfall Method

Requirements



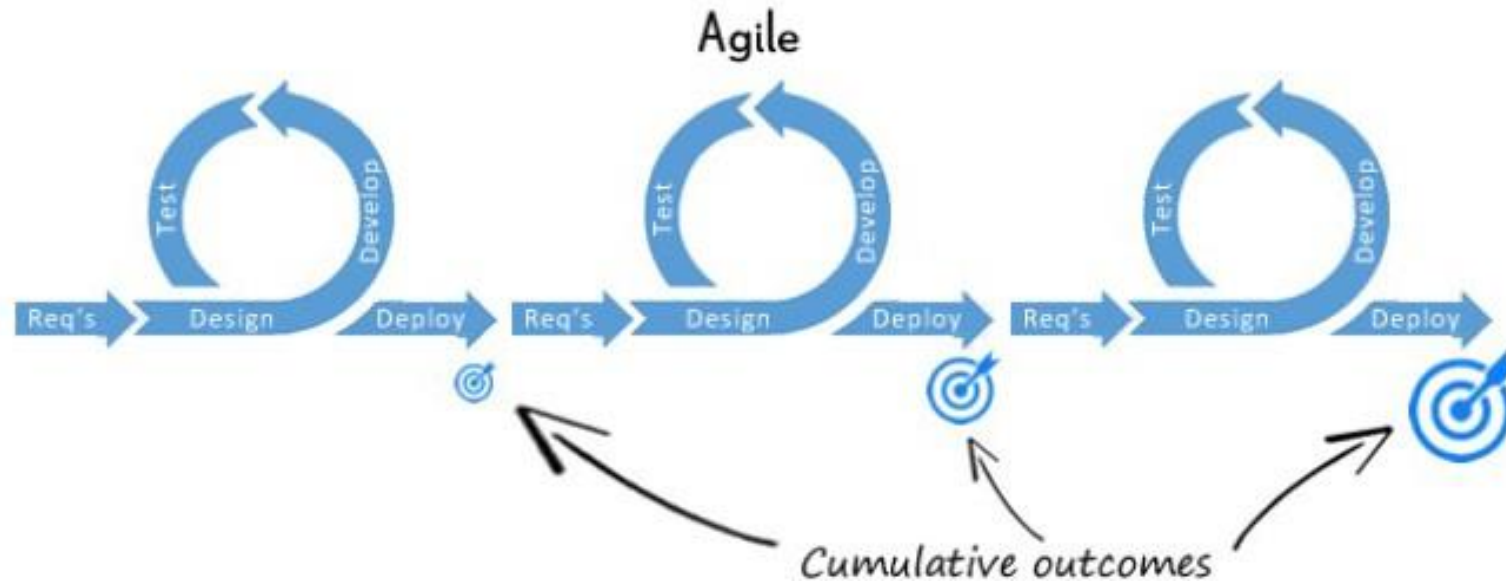
- ❑ Each activity confined to its “phase”
- ❑ Unidirectional, no way back; finish this phase before moving to the next

Spiral Method



- Develop increment of functionality, repeat in a feedback loop

Agile Method



- ❑ Continuous user feedback essential
- ❑ Feedback loops on several levels of granularity

Software Measurement

⚙️ What to measure?

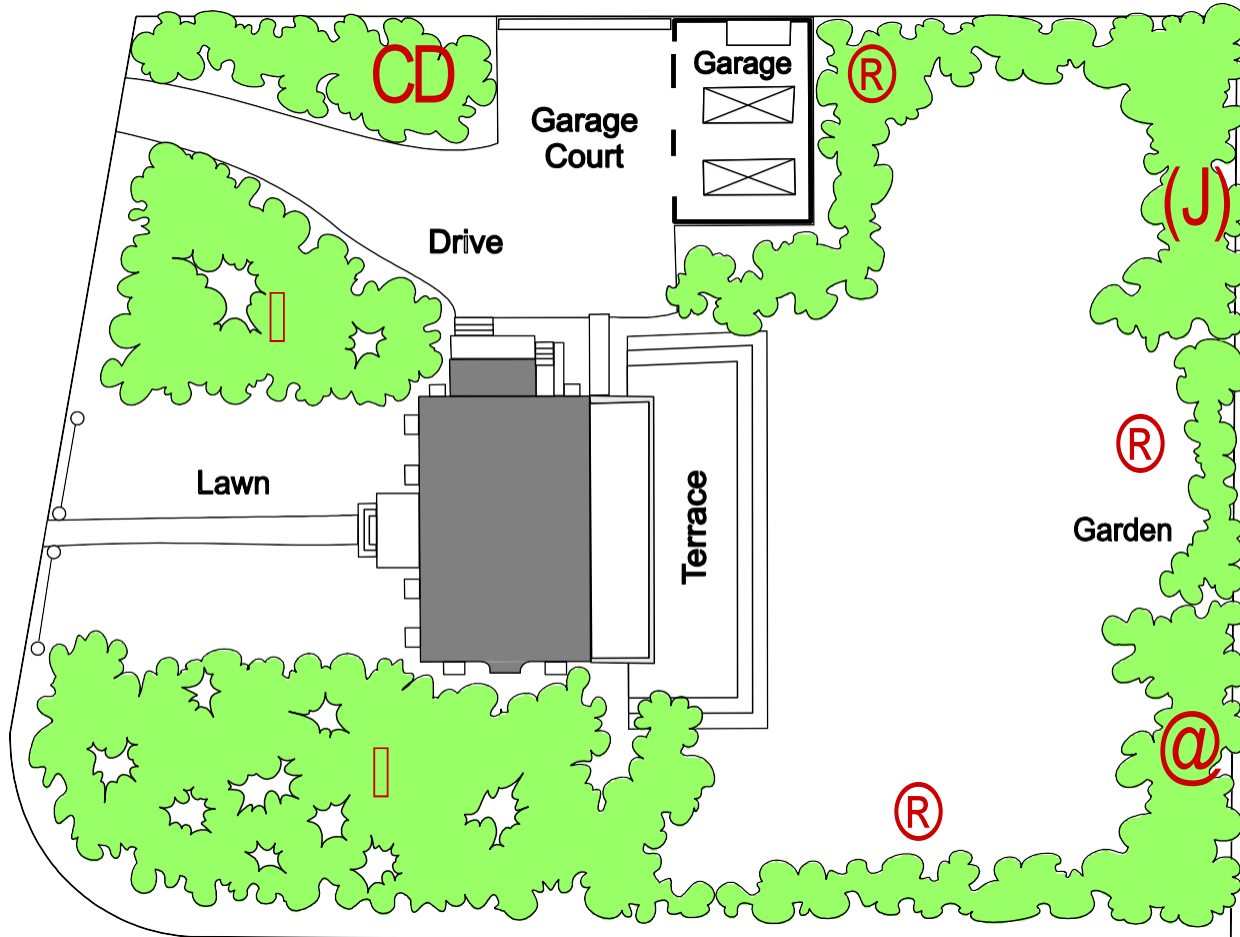
- 👉 Project (developer's work): for budgeting and scheduling
- 👉 Product: for quality assessment

Work Estimation Strategy

1. Make initial guess for a little part of the work
2. Do a little work to find out how fast you can go
3. Make correction on your initial estimate
4. Repeat until no corrections are needed or work is completed

Hedge Pruning Example

Step 1: Divide the problem into *small* & *similar* parts



Step 2: Estimate *relative* sizes of all parts

$$\text{Size}(Q) = 4$$

$$\text{Size}(\square) = 7$$

$$\text{Size}(C) = 10$$

$$\text{Size}(\textcircled{R}) = 3$$

$$\text{Size}(\textcircled{R}) = 4$$

$$\text{Size}(@) = 2$$

$$\text{Size}(<v) = 4$$

$$\text{Size}(@) = 7$$

- Step 3: Estimate the size of the total work
- Step 4: Estimate velocity of work
- Step 5: Estimate the work duration (size/ velocity)

Sizing the Problem

⚙ Assumptions:

- 👉 Relative size estimates are accurate
 - ⌘ That's why parts should be small & similar-size!

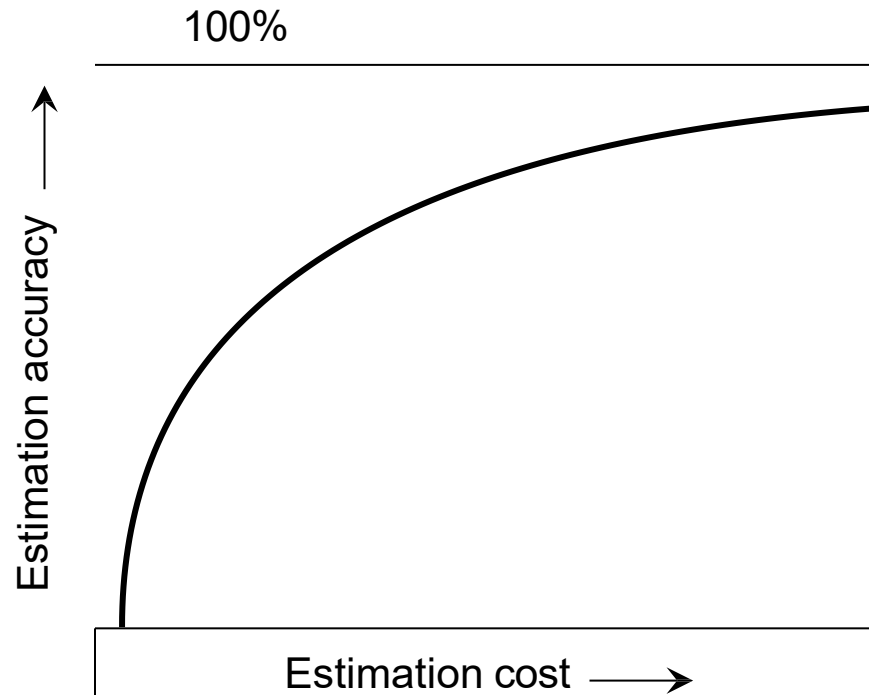
⚙ Advantages:

- 👉 Velocity estimate may need to be adjusted (based on observed progress)
- 👉 However, the total duration can be recomputed quickly
 - ⌘ Provided that the relative size estimates of parts are accurate
 - accuracy easier achieved if the parts are small and similar-size

⚙ Unfortunately:

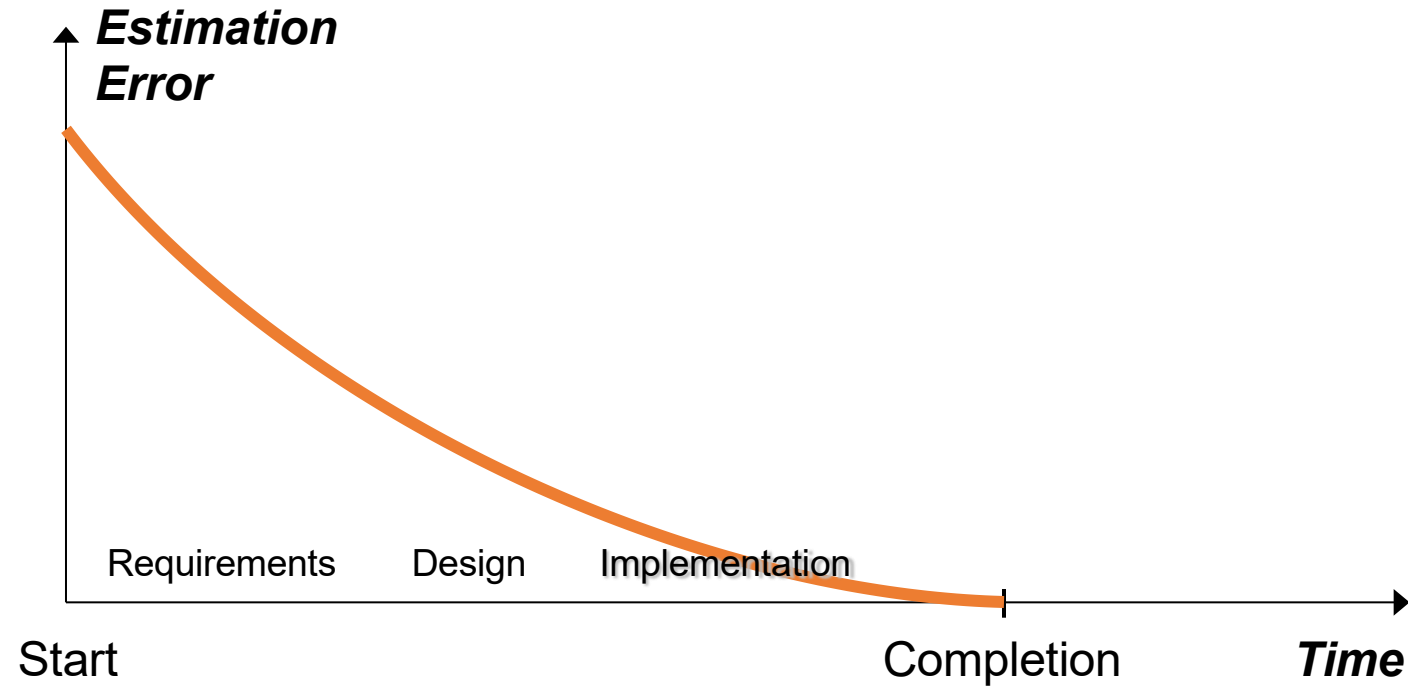
- 👉 Unlike hedges, software is mostly **invisible** and **does not exist** when project is started
 - ⌘ The initial estimate hugely depends on experience and imagination

Exponential Cost of Estimation



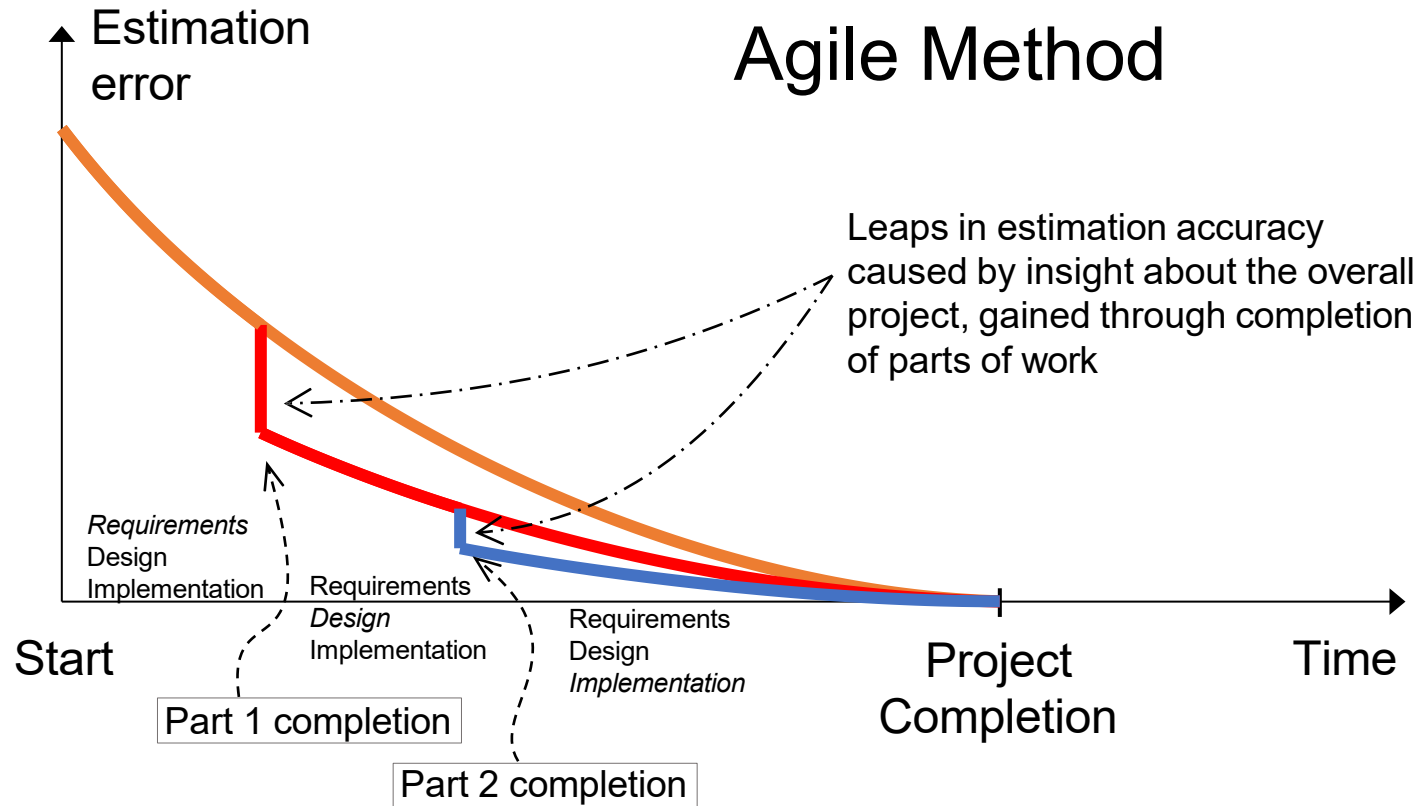
- ❑ Improving accuracy of estimation beyond a certain point requires huge cost and effort (known as the law of diminishing returns)
- ❑ In the beginning of the curve, a modest effort investment yields huge gains in accuracy

Estimation Error Over Time: Waterfall Method



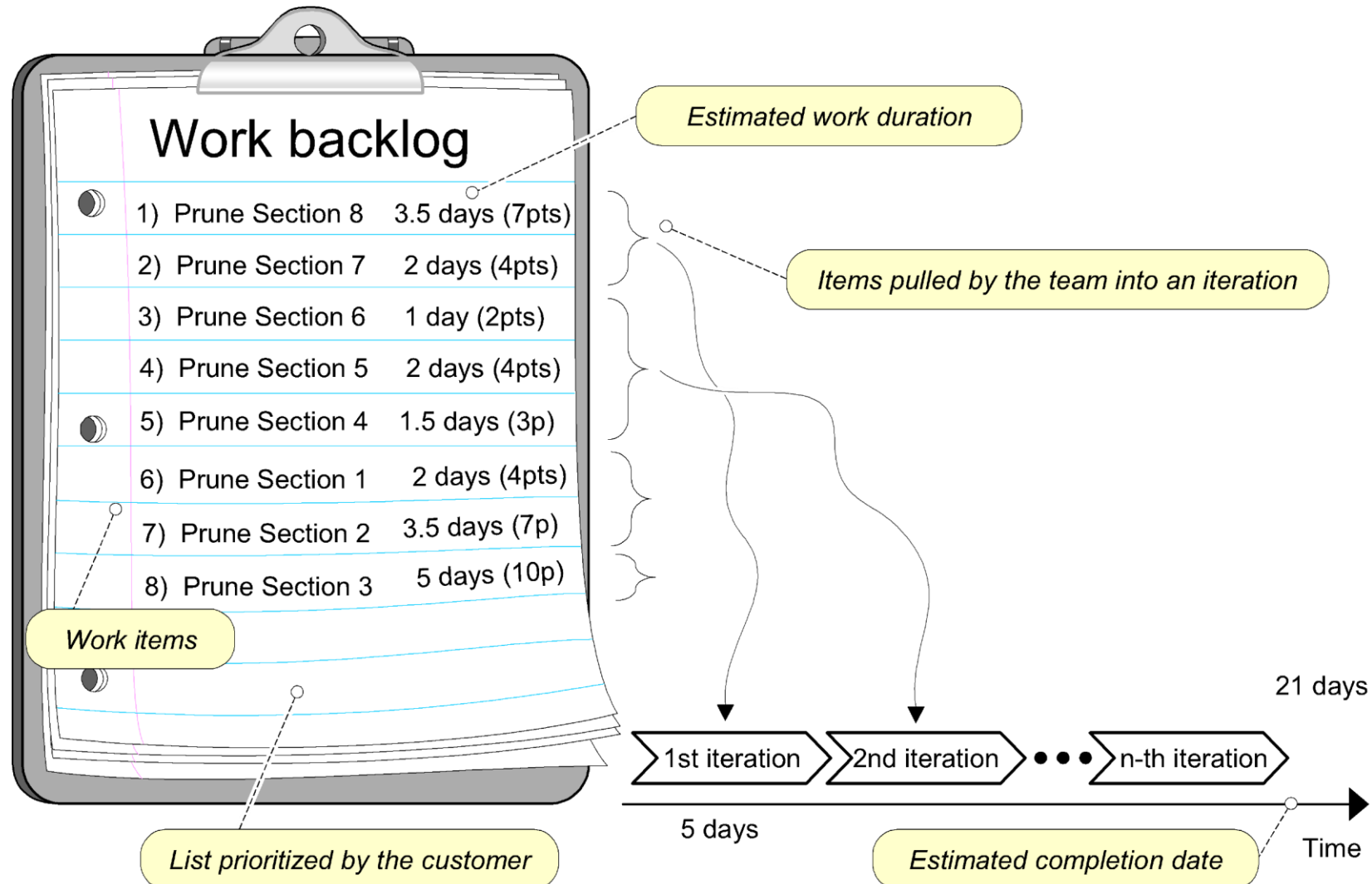
- ❑ Waterfall method *cone of uncertainty* starts high and *gradually* converges to zero as the project approaches completion

Estimation Error Over Time: Agile Method



- Agile method *cone of uncertainty* starts high and *in leaps* converges to zero as the project approaches completion

Agile Project Effort Estimation



Software Process Activities

- ⚙ **Software specification**, where customers and engineers define the software that is to be produced and the constraints on its operation.
- ⚙ **Software development**, where the software is designed and programmed.
- ⚙ **Software validation**, where the software is checked to ensure that it is what the customer requires.
- ⚙ **Software evolution**, where the software is modified to reflect changing customer and market requirements.

Software engineering fundamentals

- ⚙️ Some fundamental principles apply to all types of software system, irrespective of the development techniques used:
 - 👉 Systems should be developed **using a managed and understood development process**. Of course, different processes are used for different types of software.
 - 👉 Dependability and performance are important for all types of system.
 - 👉 Understanding and managing the software specification and requirements (what the software should do) are important.
 - 👉 Where appropriate, you should reuse software that has already been developed rather than write new software.

Internet Software Engineering

- ⚙ The Web is now a platform for running application and organizations are increasingly developing web-based systems rather than local systems.
- ⚙ **Web services** allow application functionality to be accessed over the web.
- ⚙ **Cloud computing** is an approach to the provision of computer services where applications run remotely on the 'cloud'.
 - 👉 Users do not buy software but pay according to use.

Web-based Software Engineering

- ⚙ Web-based systems are complex distributed systems but the fundamental principles of software engineering are as applicable to them as they are to any other types of system.
 - ☞ Software reuse
 - ☞ Incremental and agile development
 - ☞ Service-oriented systems
 - ✂ Software components are stand-alone web services
 - ☞ Rich interfaces
 - ✂ Using Interface development technologies such as AJAX and HTML5

Software Engineering Ethics

- ⚙ Software engineering **involves wider responsibilities than simply the application of technical skills.**
- ⚙ Software engineers must **behave in an honest and ethically responsible** way if they are to be respected as professionals.
- ⚙ Ethical behaviour is more than simply upholding the law but involves following a set of principles that are morally correct.

Issues of professional responsibility

⚙ Confidentiality

- 👉 Engineers should normally respect the confidentiality of their employers or clients irrespective of whether or not a formal confidentiality agreement has been signed.

⚙ Competence

- 👉 Engineers **should not misrepresent their level of competence**. They should not knowingly accept work which is outwith their competence.

ACM/IEEE Code of Ethics

- ⚙️ ACM: Association for Computing Machinery
- ⚙️ IEEE: Institute of Electrical and Electronics Engineers
- ⚙️ The professional societies in the US have cooperated to produce a code of ethical practice.
- ⚙️ Members of these organisations sign up to the code of practice when they join.
- ⚙️ The Code contains eight Principles related to the behaviour of and decisions made by professional software engineers, including practitioners, educators, managers, supervisors and policy makers, as well as trainees and students of the profession.

Rationale for the code of ethics

- ⚙ *Computers have a central and growing role in commerce, industry, government, medicine, education, entertainment and society at large. Software engineers are those who contribute by direct participation or by teaching, to the analysis, specification, design, development, certification, maintenance and testing of software systems.*
- ⚙ *Because of their roles in developing software systems, software engineers have significant opportunities to do good or cause harm, to enable others to do good or cause harm, or to influence others to do good or cause harm. To ensure, as much as possible, that their efforts will be used for good, software engineers must commit themselves to making software engineering a beneficial and respected profession.*

The ACM/IEEE Code of Ethics

Software Engineering Code of Ethics and Professional Practice

ACM/IEEE-CS Joint Task Force on Software Engineering Ethics and Professional Practices

PREAMBLE

The short version of the code summarizes aspirations at a high level of the abstraction; the clauses that are included in the full version give examples and details of how these aspirations change the way we act as software engineering professionals. Without the aspirations, the details can become legalistic and tedious; without the details, the aspirations can become high sounding but empty; together, the aspirations and the details form a cohesive code.

Software engineers shall commit themselves to making the analysis, specification, design, development, testing and maintenance of software a beneficial and respected profession. In accordance with their commitment to the health, safety and welfare of the public, software engineers shall adhere to the following Eight Principles:

Ethical principles

1. **PUBLIC** - Software engineers shall act consistently with the public interest.
2. **CLIENT AND EMPLOYER** - Software engineers shall act in a manner that is in the best interests of their client and employer consistent with the public interest.
3. **PRODUCT** - Software engineers shall ensure that their products and related modifications meet the highest professional standards possible.
4. **JUDGMENT** - Software engineers shall maintain integrity and independence in their professional judgment.
5. **MANAGEMENT** - Software engineering managers and leaders shall subscribe to and promote an ethical approach to the management of software development and maintenance.
6. **PROFESSION** - Software engineers shall advance the integrity and reputation of the profession consistent with the public interest.
7. **COLLEAGUES** - Software engineers shall be fair to and supportive of their colleagues.
8. **SELF** - Software engineers shall participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession.

ACS Code of Ethics

1. The Primacy of the Public Interest

You will place the interests of the public above those of personal, business or sectional interests.

2. The Enhancement of Quality of Life

You will strive to enhance the quality of life of those affected by your work.

3. Honesty

You will be honest in your representation of skills, knowledge, services and products.

4. Competence

You will work competently and diligently for your stakeholders.

5. Professional Development

You will enhance your own professional development, and that of your staff.

6. Professionalism

You will enhance the integrity of the ACS and the respect of its members for each other.

Tasks related to Assessment 1

- ⚙ How do you apply the first law of software engineering to this project? In other words, how do you understand the problem domain?
- ⚙ How do you apply the second law of software engineering to this project? In other words, how do you make users understand the software you develop?